



Git and GitHub Cheat Sheet



Your companion reference for the Git & GitHub for Beginners course. Every command you see in the videos, in one place.

1. Installation & Setup

Install Git

```
# Check if Git is installed git --version # Install Git on macOS  
(Homebrew) brew install git # Windows: download from https://git-  
scm.com
```

Install the GitHub CLI

```
# Install the GitHub CLI on macOS brew install gh # Verify the  
install gh --version # Authenticate with GitHub (browser based flow)  
gh auth login
```

- Optional: SSH key setup for GitHub

2. Configuring Git

Set your identity and defaults once so every commit is tagged correctly.

```
# Set your commit name and email git config --global user.name "Your Name" git config --global user.email "you@example.com" # Make main the default branch on new repos git config --global init.defaultBranch main # Use VS Code as your default editor git config --global core.editor "code --wait" # Inspect your global config git config --global --list
```

3. Starting a Repository

```
# Initialise a new Git repo in the current folder git init # Clone an existing remote repo git clone https://github.com/owner/repo.git # Clone into a custom folder name git clone https://github.com/owner/repo.git my-folder
```

4. The Core Daily Workflow

Check status, stage changes, commit, and push.

```
# See what has changed git status # Stage a single file git add filename.py # Stage everything in the working tree git add . # Commit the staged changes with a message git commit -m "Describe the change here" # Push commits to the remote git push # Pull remote changes into the current branch git pull # Download remote changes without merging them git fetch
```



Commit message tips: keep the subject under 50 characters, use imperative mood ("Add login form", not "Added login form"), and say what changed and why.

5. Viewing History

```
# Full commit history git log # One commit per line (compact view) git log --oneline # History for a specific branch git log origin/feature/my-branch # Commit graph with branches git log --oneline --graph --all
```

6. The Staging Area

```
# Unstage a specific file (keep the edits in working tree) git restore --staged filename.md # Unstage everything git restore --staged .
```

7. The .gitignore File

A .gitignore file tells Git which files and folders to leave untracked (logs, secrets, build artefacts, virtual environments, etc.).

```
# Credentials & secrets .env *.pem *.key credentials.json
service_account.json # Python __pycache__/.venv/ venv/
.pytest_cache/ # Node node_modules/ # Data files *.csv *.parquet
*.avro data/ output/ # IDE & editors .vscode/ .idea/ # OS files
.DS_Store Thumbs.db # Logs logs/ *.log
```



Already committed a file by mistake? Add it to .gitignore, then stop tracking it with `git rm --cached path/to/file` and commit again.

8. Branching & Merging

Use prefixes like feature/, fix/, docs/ or chore/ to keep branch names readable.

```
# List local branches git branch # Create a new branch git branch
feature/my-branch # Switch to a branch git switch feature/my-branch
# Create and switch in one step git switch -c feature/my-branch #
Rename the current branch (often used to set main) git branch -M
main # Delete a fully merged branch git branch -d feature/my-branch
# Force delete a branch git branch -D feature/my-branch # Merge
another branch into the current one git switch main git merge
feature/my-branch
```

Resolving Merge Conflicts

When Git cannot auto merge, open the conflicted files, pick the right content (removing the <<<<<<, =====, >>>>>> markers), then stage and commit. In VS Code you can use the inline Accept Current / Accept Incoming / Accept Both buttons.

```
# After editing the conflicted files git add . git commit -m
"Resolve merge conflict" # Abort the merge and go back git merge --
abort
```

Rebase

```
# Replay the current branch on top of main git switch feature/my-
branch git rebase main # After fixing conflicts during a rebase git
add . git rebase --continue # Bail out of a rebase git rebase --
abort
```

9. Working with Remotes

```
# Connect a local repo to a GitHub repo git remote add origin
https://github.com/owner/repo.git # View the configured remotes git
remote -v # Push and set upstream tracking on first push git push -u
origin main # Push a feature branch and track it git push -u origin
feature/my-branch # Update origin URL (e.g., switch from HTTPS to
SSH) git remote set-url origin git@github.com:owner/repo.git #
Remove a remote git remote remove origin
```

10. Pull Requests & Forking

Pull requests happen in the GitHub UI. Typical pattern: branch, push, open PR, review, merge, then pull main.

```
# After your PR is merged, update local main git switch main git pull # Delete the merged feature branch locally git branch -d feature/my-branch
```

Forking (open source workflow)

```
# Forking is done in the GitHub UI # Then clone your fork git clone https://github.com/your-user/repo.git # Add the original project as 'upstream' git remote add upstream https://github.com/original-owner/repo.git # Keep your fork in sync git fetch upstream git switch main git merge upstream/main git push
```

11. Inspecting Changes with Diff

```
# Unstaged changes (working tree vs index) git diff # Staged changes (index vs last commit) git diff --staged # Compare two branches git diff main..feature/my-branch # Diff for a single file git diff path/to/file.py
```

12. Discarding Changes

```
# Discard edits in a specific file (restore from last commit) git  
restore filename.py # Discard all unstaged edits in the working tree  
git restore . # Remove an untracked file rm filename.py
```

13. Stashing

Stash lets you shelve uncommitted work so you can switch branches or pull cleanly.

```
# Stash your current changes git stash # Stash with a label git  
stash push -m "wip: login form" # See all stashes git stash list #  
Reapply the most recent stash and remove it git stash pop # Reapply  
without removing it git stash apply # Drop a specific stash git  
stash drop stash@{0} # Clear all stashes git stash clear
```

14. Amending, Reverting & Undoing Commits

```
# Edit the most recent commit (message or staged changes) git commit  
--amend -m "Updated message" # Create a new commit that undoes an  
earlier commit git revert <commit-hash> # Move HEAD back one commit,  
keep changes staged git reset --soft HEAD~1 # Move HEAD back one  
commit, keep changes in working tree git reset --mixed HEAD~1 # Move  
HEAD back one commit and drop all changes (destructive) git reset --  
hard HEAD~1
```



Rule of thumb: use revert on shared branches (safe, does not rewrite history). Use reset only on local branches that have not been pushed.

15. Renaming a Branch

```
# Rename the branch you are currently on git branch -M new-name #  
Rename another branch git branch -m old-name new-name # Push the  
renamed branch and reset upstream git push -u origin new-name #  
Delete the old name on the remote git push origin --delete old-name
```

16. Branching Strategies (Quick Reference)

- Feature branching: one branch per feature or fix, merged into main via pull request.
- GitHub Flow: main is always deployable, short lived feature branches, merge via PR, deploy on merge.
- Git Flow: long lived develop and main branches, plus feature, release and hotfix branches. Suits versioned releases.
- Trunk based development: small commits straight to main (or very short branches) behind feature flags.

17. End-to-End Workflow Recap

1. Create a branch: `git switch -c feature/my-branch`
2. Make code changes
3. Stage: `git add .`
4. Commit: `git commit -m "Descriptive message"`
5. Push: `git push -u origin feature/my-branch`

6. Open a pull request on GitHub and request a review
7. Address review comments and push again
8. Merge the PR on GitHub
9. Sync locally: `git switch main` && `git pull`
10. Tidy up: `git branch -d feature/my-branch`

18. Useful Aliases

Save typing with a few common shortcuts.

```
git config --global alias.st status git config --global alias.co  
checkout git config --global alias.br branch git config --global  
alias.cm "commit -m" git config --global alias.lg "log --oneline --  
graph --all"
```

19. Common Errors & Fixes

- **fatal: not a git repository** — you are not inside a repo. Run `git init` or `cd` into one.
- **rejected – non-fast-forward** on push: the remote has commits you do not have. Run `git pull --rebase` then push again.
- **Please tell me who you are** on commit: set `git config --global user.name/user.email`.
- **Your branch is ahead of 'origin/main' by N commits** — you have local commits not yet pushed. Run `git push`.
- **detached HEAD**: you checked out a specific commit. Create a branch with `git switch -c new-branch` to save work.
- **merge conflict**: open conflicted files, resolve the markers, then `git add` and commit.

20. Further Learning

- Official Git documentation — <https://git-scm.com/doc>
- GitHub Skills (free interactive courses) — <https://skills.github.com>

© 2024 GitHub, Inc. All rights reserved.